

Transazioni

Proprietà ACID

Atomicità, coerenza, isolamento e durabilità spiegati senza formalismi inutili.

ACID è un modo rapido per dire: questa transazione è affidabile

ACID è una sigla usata per descrivere le proprietà attese da una buona transazione.

Pensa a una consegna importante. Vuoi che parta tutta, arrivi nel posto giusto, non si mescoli con altre consegne e resti registrata dopo la conferma.

- **Atomicità:** o tutto o niente
- **Coerenza:** i dati restano validi
- **Isolamento:** le transazioni non si disturbano troppo
- **Durabilità:** dopo il commit, il dato resta salvato

Un significato alla volta

Atomicità vuol dire che non vuoi mezze operazioni. Se trasferisci denaro, non vuoi togliere soldi da un conto senza aggiungerli all'altro.

Coerenza vuol dire che i dati devono rispettare le regole del database. Per esempio, un ordine non dovrebbe puntare a un cliente inesistente.

Isolamento vuol dire che più operazioni contemporanee non devono pestarsi troppo i piedi.

Durabilità vuol dire che, dopo la conferma, il risultato non deve sparire.

Perché dovresti ricordartelo

Non serve recitarlo come una poesia a memoria. Ti serve come bussola mentale.

Quando studi transazioni, concorrenza o affidabilità del database, **ACID ti ricorda che l'obiettivo non è solo eseguire query, ma eseguirle bene anche sotto stress.**

:::note ACID non è un comando SQL. È un modo per descrivere il comportamento affidabile che ti aspetti dalle transazioni. :::

Deadlock

Quando due transazioni si bloccano a vicenda e come capirlo.

Qui il problema non è il carico: è l'incastro sbagliato

Un deadlock succede quando due transazioni si aspettano a vicenda e nessuna può proseguire. È come due persone ferme in un corridoio stretto, ognuna in attesa che si sposti l'altra.

Un esempio mentale semplice

- la transazione A blocca la riga 1 e poi vuole la riga 2
- la transazione B blocca la riga 2 e poi vuole la riga 1

Risultato: si fermano entrambe.

Cosa fa il database

Molti database sanno rilevare questa situazione e annullano una delle due transazioni per sbloccare il sistema.

Questo non significa che il database "ha perso dati" a caso. Di solito annulla una transazione, segnala l'errore e lascia all'applicazione il compito di riprovare se ha senso.

Come ridurre il rischio

Puoi abbassare la probabilità di deadlock se:

- tieni le transazioni brevi
- accedi alle tabelle sempre nello stesso ordine
- eviti operazioni inutilmente lunghe mentre i lock sono attivi

Capire i deadlock è importante perché ti insegna che la concorrenza non è magia: è coordinazione.

Un'abitudine pratica

Se due parti del programma modificano sempre `clienti` e poi `ordini`, è meglio che tutte seguano quell'ordine.

Quando ogni pezzo del sistema prende le risorse nello stesso ordine, gli incastri diventano meno probabili.

Livelli di isolamento

READ COMMITTED, REPEATABLE READ, SERIALIZABLE e gli altri livelli principali.

Quando più persone lavorano insieme, il problema non è solo cosa fai ma cosa vedi

I livelli di isolamento stabiliscono quanto una transazione può vedere del lavoro delle altre transazioni in corso.

Se due persone modificano o leggono dati nello stesso momento, il database deve decidere quanta "privacy" dare a ciascuna operazione.

Pensa a due persone che aggiornano lo stesso documento. Possono vedere subito le modifiche dell'altra? Devono aspettare? Il sistema deve scegliere una regola.

Livelli noti:

- `READ UNCOMMITTED`
- `READ COMMITTED`
- `REPEATABLE READ`
- `SERIALIZABLE`

La regola intuitiva

Più isolamento significa di solito più sicurezza e prevedibilità, ma anche meno libertà di lavorare in parallelo.

È sempre un equilibrio tra protezione e prestazioni.

Come pensarli senza perdersi

Non è indispensabile memorizzare subito i dettagli fini di ogni livello. All'inizio ti basta capire questo:

- i livelli bassi lasciano più libertà
- i livelli alti proteggono di più dai comportamenti strani

Una scala mentale utile

READ COMMITTED è una scelta comune: una transazione vede solo dati già confermati da altre transazioni.

REPEATABLE READ protegge di più la lettura: se rileggi gli stessi dati nella stessa transazione, ti aspetti più stabilità.

Il più rigoroso

SERIALIZABLE è il livello più severo tra quelli classici. Fa comportare le transazioni come se passassero una alla volta.

È molto protettivo, ma può essere più costoso in termini di concorrenza.

:::caution I dettagli precisi cambiano tra database. Prima impara l'idea generale, poi controlla il comportamento del database che stai usando. :::

Locking e concorrenza

Come il database gestisce accessi simultanei agli stessi dati senza farli scontrare.

Quando due mani cercano lo stesso oggetto nello stesso momento

La concorrenza nasce quando più utenti o processi lavorano sugli stessi dati nello stesso momento. Il database deve evitare che si creino conflitti o risultati incoerenti.

Un **lock** è come il cartello "occupato" appeso a una porta. Dice agli altri: aspetta un attimo, questa parte è già in uso.

In SQL, quel cartello può riguardare una riga, una tabella o altre parti interne del database.

Perché servono i lock

Senza meccanismi di locking, due aggiornamenti contemporanei potrebbero:

- sovrasciversi a vicenda

- leggere dati a metà
- produrre risultati difficili da prevedere

La cosa importante da capire

Il locking non è un difetto. È una protezione. Il problema nasce solo quando i lock durano troppo o vengono gestiti male.

Per questo conviene scrivere transazioni brevi e mirate, invece di lasciarle aperte inutilmente.

Un esempio mentale

Se due casse del supermercato provano a vendere l'ultimo pezzo dello stesso prodotto nello stesso momento, serve una regola. Il database deve evitare che entrambe dicano "venduto" come se il pezzo fosse doppio.

I lock aiutano a coordinare questi momenti delicati.

:::tip Quando una query resta bloccata, non pensare subito a un errore. Potrebbe essere in attesa che un'altra transazione finisca. :::

Transazioni

BEGIN, COMMIT e ROLLBACK per trattare più operazioni come un unico blocco.

Alcune operazioni devono riuscire insieme oppure fallire insieme

Una transazione è un gruppo di operazioni che il database tratta come un solo blocco logico. Questo è fondamentale quando uno stato "a metà" sarebbe pericoloso o incoerente.

Pensa a un trasferimento di denaro: togliere soldi da un conto e aggiungerli a un altro deve funzionare tutto insieme.

Un esempio concreto

```
BEGIN;  
UPDATE conti  
SET saldo = saldo - 100  
WHERE id = 1;  
  
UPDATE conti  
SET saldo = saldo + 100  
WHERE id = 2;  
  
COMMIT;
```

Qui il lavoro inizia con `BEGIN` e si conclude con `COMMIT`.

Il ruolo dei tre comandi chiave

- `BEGIN` : inizia la transazione
- `COMMIT` : conferma tutto in modo definitivo
- `ROLLBACK` : annulla le modifiche della transazione

Quando entra in gioco **ROLLBACK**

Se qualcosa va storto a metà, puoi fare marcia indietro:

```
ROLLBACK;
```

Questa è la grande forza delle transazioni: ti aiutano a evitare stati rotti o incompleti.

Come pensarla nella vita reale

Immagina di spedire un pacco. Non basta stampare l'etichetta: devi anche pagare la spedizione e consegnarlo al corriere.

Se uno di questi passaggi fallisce, non vuoi lasciare tutto in uno stato confuso. Vuoi tornare indietro e riprovare con ordine.

:::tip Tieni le transazioni brevi. Più restano aperte, più possono bloccare il lavoro di altre operazioni. :::